

Contents

PLAYING GOD — DESIGN BRIEF v3

1. WHAT THIS BUILDS

2. INVARIANTS

3. THE GENOME

4. SYNTHESIS

5. INITIALISATION AND PRIORS

6. VARIATION

7. SEARCH — MAP-ELITES

8. FITNESS AND MEASUREMENT INTEGRITY

9. RENDER LENGTH SERVO

10. THE PREDICTOR

11. VISUALISER

12. IMPLEMENTATION

13. BUILD ORDER AND GATES

14. LOGGING

15. EVALUATION PROTOCOL

16. DEFERRED

APPENDIX — CONSTANTS

1

1

2

5

7

8

10

15

19

22

24

25

26

26

29

31

32

33

PLAYING GOD — DESIGN BRIEF v3

Complete system specification. Every value stated is a decision. Sections are ordered so that an implementer, or a model generating component sub-briefs, can work top to bottom.

1. WHAT THIS BUILDS

A procedural sound generator in which sounds are evolved rather than designed.

The unit is a **Creature**: the audible expression of a **genome**. Genomes describe a bank of oscillators that can modulate one another. Sound is produced by rendering the genome to samples.

Fitness is **measured listening time and nothing else**. No ratings, no rankings, no stated preference. A listener presses a key; a clock runs; the next keypress stops it; that duration is the entire fitness signal.

The interface is a single page carrying almost nothing:

Key	Action
SPACE	Stop current Creature (8 ms fade), draw a new one, replace visuals
M	Stop current Creature, play a mutated child of it
B	Stop current Creature, return to its parent and replay

The keyboard handler must be table-driven so further bindings can be added without restructuring.

Search is **MAP-Elites**: a 256-cell archive over two perceptual axes, each cell holding the best Creature found for that kind of sound. One listen is one evaluation and one archive update. There is no generational structure.

A learned model (**the Predictor**) later predicts dwell time from a genome, allowing many iterations to run with no listener present.

Read §13 before building anything. This system is built in four stages separated by acceptance gates, and the gates are not review points — they are stop conditions. Three of them test assumptions the downstream design depends on, and if one fails, everything built after it is wasted work. Do not build the whole system and evaluate afterwards.

Instrumentation is not optional and is specified in §14. It must be built alongside each stage, not retrofitted. An evaluation is only as good as what was recorded while the system ran, and what needs recording is determined by the gates and invariants rather than by what seems interesting at the time.

2. INVARIANTS

Five rules. Every design decision, including future ones, must be checkable against them.

2.1 Priors bias sampling; they never truncate the space

The premise of the system is that the target is unknown and unspecifiable in advance. Search space is therefore an asset, not a cost.

Dimensionality and cardinality are different numbers and must never be confused.

- **Dimensionality: 6,102.** The number of parameters in a genome. This is the figure quoted throughout the document, and it is what determines search cost.
- **Cardinality: about $10^{14,695}$.** The number of distinct genomes, at a reference quantisation of 8 bits per parameter: $2^{(8 \times 6,102)} = 2^{48,816} \approx 10^{14,695}$.

For scale, the observable universe contains on the order of **10^{80}** atoms. The genome space is larger by a factor of about $10^{14,615}$.

The figure moves with the quantisation assumed and should always be quoted with it: at 10 bits per parameter it is $\sim 10^{18,369}$, at ~ 6.6 bits (roughly 100 distinguishable values each) $\sim 10^{12,123}$. Every such figure is an over-count of *audibly distinct* Creatures, because the genome→sound map is many-to-one by design (§3.1) — but the over-count is a rounding error at these magnitudes.

Even the **expressed** subspace at initialisation — ~ 163 parameters — is about 10^{393} at 8 bits, still unenumerable by many hundreds of orders of magnitude.

This is the concrete content of the invariant rather than a decoration on it. Nothing here will ever be searched exhaustively, so the only question that matters is what the search is *steered toward and away from*. A constraint that removes a region does not make the space tractable — it cannot, at these magnitudes — it only guarantees that whatever was in the removed region is never found. Every prior in this document is a bias on **initial sampling and mutation recentring only**. No prior may make any region of the declared space unreachable.

Implementer check: for each prior, ask whether it makes some region *unreachable* or merely *unlikely*. Unreachable is a defect.

Declared gene ranges are not priors — they define the space. Where a range is stated below it is deliberately wide.

Scope of the rule — this matters and is easy to get wrong. “Reachable” must be judged at the level of **operators, mechanisms and regions**, never at the level of individual outcomes.

In a space this size every particular genome is astronomically improbable. That is what a large space *is*. Applying a rarity test to individual Creatures would condemn the project on its first line.

Level	In scope?	Examples
Operator / mechanism — a kind of move the system can make	Yes	crossing with a genome at twice the median distance; unmuting a wave; rerouting a modulation source; a three-partner recombination
Region — a coarse structural or behavioural class	Yes	genomes with 20+ active waves; genomes containing feedback loops; a behaviour-space cell

Level	In scope?	Examples
Point — one specific genome	No	any particular 6,102-parameter Creature

The operational test. Take the rate at which the thing occurs, multiply by the number of attempts the system will make over its lifetime, and ask whether the expected count is at least of order 1. For a mechanism, “attempts” is the number of times it could fire — crossover events are roughly $0.5 \times \text{listens}$. For a region, it is the number of candidate generations.

Applied to the partner kernel (§6.6): distant pairings occur often enough that, over a few thousand crossover events, they happen tens of times. That is rare and real. At the previous selectivity they would have happened perhaps once, which is rare and notional.

The question to ask is “*would this system, run to its budget, ever do this at all?*” — never “*would this system ever produce this exact Creature?*”, which is the wrong question with a known answer.

This system is the deliberate opposite of a hand-tuned instrument in which wrong notes are unreachable by construction. Any proposal that improves output by narrowing what can be produced is rejected on principle, not on merit.

2.2 Priors may be biased. Instruments may not.

A prior is an admitted, deliberate bias about where to look first. That is legitimate and unavoidable — sampling has to come from some distribution.

The **instruments** are different: the fitness function, the behaviour descriptors, the archive geometry, the render-length servo, the diagnostics. These measure and organise. They must carry no assumption about what will score well. An instrument that encodes a prediction will find that prediction, and the system will have confirmed the designer’s taste rather than discovered anything.

Priors decide where to look. Instruments must not decide what counts as having found something.

2.3 Designer assumptions are the recurring failure mode on this project

Three times, an assumption the system exists to avoid was written into what should have been neutral machinery. All three were caught only on review, and all three looked like reasonable engineering on the page.

As a claim about what will score well:

- The temporal-development archive axis was justified as “the descriptor most aligned with what dwell time measures” — a prediction that sounds which change over time will win.
- A bimodal dwell distribution was declared the signature of a healthy population — a prediction that a mix of fast skips and long listens is what success looks like.

As a limit on what can be reached:

- Crossover was gated on a hard compatibility threshold $D < 0.25$, with a fallback when no partner qualified. That made every more-distant pairing *unreachable* rather than merely unlikely — a truncation of which combinations could ever be tried. Replaced by a soft decaying kernel (§6.6).

None of it is known. Change over time might sustain attention through variety or might be chaotic and skipped instantly; a drone might be tedious or might be transcendent; a pairing between two distant genomes might be worthless or might be the only route to combining two independently discovered structures. **We do not know. That is the entire premise.**

Three instances is a pattern, not a slip, and the two forms need checking separately. When adding to this document, ask both questions:

1. *Does this claim to know what will score well?*
2. *Does this make anything unreachable rather than merely unlikely?*

A yes to either means the decision needs a different justification, or the decision is wrong.

2.3b Listens are the scarce resource

A listen is the only way fitness enters the system. Everything else — genomes, renders, model evaluations — is effectively free. The binding constraint on this project is therefore the number of listens available, and every mechanism must be assessed by what it costs in them.

Two consequences that recur below:

- A mechanism that consumes listens must earn them against the alternative use of the same listens for generating new Creatures. This is why the explicit replay branch was removed once deep cells (§7.2) made implicit averaging available.
- The cost of a bad decision inside the search is usually **a wasted listen**, not damage to the population. Under a deep-grid archive a poor Creature occupies at most one of D slots in one cell and is evicted in time. It does not propagate the way a poor recombinant does in a generational GA. Protections against disruption should therefore be justified by evaluation economy, not by harm prevention, and are correspondingly cheaper to relax (§6.4).

Repeated exposure is also a cost: hearing the same Creature several times in short succession measures novelty exhaustion rather than the sound. Constraint in §8.5.

2.4 Representation discipline

Three representations. Every subsystem declares which it operates on.

Tier	Form	Cost	Operated on by
GENOME	~6,100 numbers	free	mutation, crossover, the Predictor, the visualiser's slow channel
SAMPLES	Float32Array, one entry per sample	~1 ms per rendered second	behaviour descriptors, perceptual distance, the visualiser's fast channel
AUDIO	samples handed to an output device	requires a listener	playback only

GENOME → SAMPLES is deterministic and many-to-one; it cannot be inverted. Nothing may attempt to recover a genome from samples.

The Predictor operates on GENOME only and carries a second output head predicting the behaviour descriptor, so that autonomous operation performs no renders at all.

2.5 No gene may touch the fitness function

Genes may influence **variation** (mutation rates, step sizes, recombination behaviour). Genes may not influence **how fitness is measured or accumulated**.

A gene controlling its own measurement is selected on its effect on measurement rather than on quality. A lineage-depth gene would be driven toward shallow averaging because higher variance yields more chances of a fluke high score; a render-length gene would be driven longer because longer renders make more dwell available.

Consequences, all enforced below: lineage depth is global; render length is global; archive axes are global; attention gating is global.

Recombination-behaviour genes are permitted, with one exclusion: a gene weighting **its own carrier's** contribution to offspring is forbidden. Such a gene is under direct selection to maximise itself irrespective of quality — the mechanism is segregation distortion, a reproduction-channel exploit rather than a fitness gain.

2.6 Declared space is vast; expressed space starts small

The genome declares ~6,100 parameters. At initialisation roughly **163** affect the output. The remainder is neutral material — muted waves, unused envelope nodes — which mutates freely at zero fitness cost and remains available for reactivation.

This is what reconciles §2.1 with tractability. Search cost scales with **expressed** parameters, not declared ones. Complexity is not granted; it is unmuted by selection when it earns its keep, and only then starts costing listens.

Nothing may prune, garbage-collect or normalise away neutral material. It is the substrate of the search.

3. THE GENOME

64 wave slots × 95 genes + 22 global = 6,102 parameters.

All genes are stored internally as floats in [0, 1] and mapped to their declared range on read. Mutation operates in stored space. Bounds are handled by reflection, not clamping.

3.1 Per-wave genes (95)

Structural — 3

Gene	Type	Notes
active	binary	If false the wave is not computed at all.
gain_out_on	binary	If false the wave contributes nothing to the audio output.
gain_mod_on	binary	If false the wave's output is unavailable as a modulation source.

Independent by design. A wave with gain_out_on false and gain_mod_on true is a pure modulator — an LFO or an FM operator — and must be inaudible in its own right. The reverse is a pure carrier. Without the split, every modulator is also a drone and low-frequency modulation is unusable.

Shape — 8

shape_sine, shape_triangle, shape_saw, shape_square (continuous, 0-1) and a binary switch for each.

Output waveform is the sum of enabled shapes weighted by their values, normalised by the sum of enabled weights. All disabled, or all weights zero, silences the wave.

Many gene combinations produce identical waveforms. This redundancy is retained deliberately: it forms neutral networks in genotype space along which a population can drift at no fitness cost, reaching phenotypes unreachable by direct hill-climbing. Do not add a canonicalisation step.

Timing — 5

Gene	Encoding	Range
pre_wait	log	0 - 30 s
duration	log	0.5 ms - 120 s
mid_wait	log	0.5 ms - 30 s
mid_wait_on	binary	off = play once, never repeat
phase	linear, wrapping	0 - 1

This trio is load-bearing and must not be simplified. A wave with duration 40 ms and mid_wait 460 ms is a rhythmic event at 2 Hz, not a tone. Several waves with related mid-waits produce polyrhythm with no rhythm machinery anywhere in the system.

Gains — 2

Gene	Encoding	Range
gain_out	dB	−80 - +6
gain_mod	log	0 - 32

One shared amplitude envelope shapes the wave’s activity; gain_out and gain_mod scale that shaped signal independently for the two destinations.

Amplitude envelope — 34

Gene	Type	Range
amp_env_on	binary	off = flat at unity
amp_env_n_nodes	integer	2 - 8
amp_node[0..7].level	dB	−80 - +24
amp_node[0..7].time	proportion	normalised across active nodes
amp_node[0..7].curve	continuous	−1 - +1
amp_node[0..7].tension	continuous	0 - 1

No separate fade-in and fade-out. One envelope with up to 8 nodes covers both and everything between, eliminating overlap ambiguity. Node times are stored as proportions and normalised by the sum across *active* nodes, so mutating one time redistributes the others rather than sliding the envelope off the end.

Nodes beyond amp_env_n_nodes are retained, inherited and mutated — neutral material per §2.6.

Curve shape is a single continuous parameter, evaluated as $y = x^{(2^{curve})}$ with tension controlling asymmetry. Convex, linear and concave lie on one continuum, so one parameter covers the family smoothly and with better locality than blended alternatives would.

Pitch — 35

Identical structure to the amplitude envelope, plus:

Gene	Encoding	Range
pitch_master	cents above 0.01 Hz	0 - 25,100 (0.01 Hz - 20 kHz)

Pitch envelope node levels are offsets in cents, range $\pm 9,600$.

Cents rather than Hz gives perceptually even mutation steps, and makes the same gene an oscillator or a sub-audio modulator depending on value.

Modulation — 6

Gene	Type	Range
pm_source	integer	0 - 63
pm_depth	log	0 - 32
pm_on	binary	
am_source	integer	0 - 63
am_depth	log	0 - 8
am_on	binary	

Per-wave meta — 2

Gene	Range	Init
sigma_wave	0.002 - 0.5	0.05
p_mutate_wave	0 - 1	0.3

3.2 Global genes (22)

fundamental_cents, tempo_bpm (30 - 300), master_gain, sigma_global, p_duplicate, p_switch_flip_scale, n_partners (1 - 8, init 1.4), partner_influence (0 - 0.5, init 0.15), mutation_fraction (0 - 1), plus **14 visualiser genes** (§11).

n_partners, partner_influence and mutation_fraction are recombination-behaviour genes, permitted under §2.5: they influence variation, not measurement. Both partner genes govern the intake of *other* parents' material and never weight the carrier's own contribution, which keeps them clear of the segregation-distortion exclusion.

n_partners and partner_influence are orthogonal — how many partners, and how much comes from them in total (§6.8). Both must be logged over time. Their drift is the system's own experimental readout on whether recombination helps in this encoding, and on what shape of recombination helps.

3.3 Kill switches

Per wave: active, gain_out_on, gain_mod_on, 4 shape switches, mid_wait_on, amp_env_on, pitch_env_on, pm_on, am_on = **12 per wave, 768 total**.

Switches are the mechanism by which structure changes discretely. A kill switch on the amplitude envelope is what makes a no-envelope wave reachable at all — under a continuous draw, a zero-length fade has measure zero and would never occur.

Switch mutation rates are calibrated per class from the locality test (§13.2), not set uniformly.

4. SYNTHESIS

4.1 Phase modulation, not frequency modulation

Modulation of pitch is implemented as phase modulation. For sinusoidal modulators the two are equivalent up to a phase shift, but under true FM the perceived pitch drifts as modulation index changes, so a mutation to depth moves two perceptual axes at once. Under PM it moves one. A locality requirement, not a stylistic choice.

4.2 Per-sample evaluation

For wave i at sample n :

```

pitch_cents = pitch_master[i] + pitch_env[i](n) · pitch_env_on[i]
freq_hz     = 0.01 · 2^(pitch_cents / 1200)
phase[i]    += freq_hz / SAMPLE_RATE

env         = amp_env_on[i] ? envelope[i](n) : 1.0
gate        = gate[i](n)                      // pre_wait / duration / mid_wait
mod_phase   = pm_on[i] ? pm_depth[i] · modsig[pm_source[i]](n - δ) : 0
raw         = shape[i](phase[i] + mod_phase)
am          = am_on[i] ? max(0, 1 + am_depth[i] · modsig[am_source[i]](n - δ)) : 1

activity    = raw · env · gate
modsig[i]   = activity · gain_mod[i] · gain_mod_on[i] · active[i]
out[i]      = activity · dB2lin(gain_out[i]) · gain_out_on[i] · active[i] · am

```

gate[i](n) is 0 before pre_wait, 1 for duration samples, then 0 for mid_wait samples, repeating while mid_wait_on.

Final output is $\sum out[i]$, peak-normalised, scaled by master_gain.

4.3 Routing, cycles and evaluation order

Routing is encoded as two source slots per wave (integer index, depth, switch) rather than an adjacency matrix. Two slots is 6 genes per wave against 8,192 for two full 64×64 matrices, and a routing mutation is a single discrete jump rather than a diffuse change across a matrix row.

At genome-compile time, run a depth-first search over the routing graph, mark back-edges, and give every back-edge a one-sample delay ($\delta = 1$; $\delta = 0$ for forward edges). Evaluate forward edges in topological order, per sample.

Cycles are permitted, including self-modulation ($pm_source[i] == i$), which produces saw-like and noise-like spectra depending on depth.

Rule — depth attenuation on reroute. Whenever pm_source or am_source changes value, by mutation or by recombination repair, the corresponding depth is multiplied by **0.05**. A rerouted connection arrives quietly and grows only if selection favours it. Without this rule, rerouting is the most destructive operation in the system and the encoding will behave brittly for that reason alone.

4.4 Modulation and envelopes coexist

Both the parametric envelope and the modulation input apply, each under its own switch, summed in the appropriate domain (additively in cents for pitch, multiplicatively for amplitude). They operate at different timescales, and a toggle between them would disconnect regions of the space that mutation should be able to walk between.

5. INITIALISATION AND PRIORS

All priors govern the initial draw and mutation recentring only. None restricts reachability (§2.1). They are admitted bias about where to look first, not claims about what will score (§2.2).

Wave activation. Each slot's active is set true with probability **0.03**, with a floor of 1 forced-active slot. Expected active waves at initialisation: ~ 2.9 .

Destination switches, per active wave. $gain_out_on$ true with probability 0.75; $gain_mod_on$ true with probability 0.5. Independent, so pure modulators arise from the start.

Pitch. Mixture: - 0.65 — harmonic: $fundamental_cents + 1200 \cdot \log_2(r)$, r drawn from $\{1, 2, 3, 4, 5, 6, 7, 8, 1/2, 1/3, 1/4, 3/2, 5/4, 5/3\}$ with probability $\propto 1/r$. - 0.25 — uniform over the **entire** declared range, 0 - 25,100 cents. - 0.10 — log-uniform in 0.01 - 20 Hz.

The uniform component covers the full range rather than an audible sub-band, so no pitch is unreachable at initialisation.

Amplitude. $gain_out$ drawn so linear amplitude $\propto 1/k$ for the k -th active wave sorted by pitch.

Time. With probability 0.6, pre_wait and mid_wait snap to $(60 / tempo_bpm) \times q$ for $q \in \{1/4, 1/3, 1/2, 2/3, 1, 3/2, 2, 3, 4\}$; otherwise log-uniform over the full range.

Duration. Bimodal: 50% log-uniform in 5-200 ms, 50% log-uniform in 0.5-120 s.

Modulation. pm_on true with probability 0.35, am_on with 0.2. Initial pm_depth log-uniform in 0.1-2.0.

Envelope node counts. Drawn from $\{2, 3\}$; free to grow to 8 by mutation.

Seeded initial batch. The archive is seeded from **32** genomes: one drawn at random as above, and 31 mutations of it at $\sigma = 0.2$. A near-identical starting batch improves early archive coherence and gives recombination homologous material to work with, without removing the variance selection needs. Do not seed with 32 independent random genomes.

Ordering of the seed batch. §7.5 requires a genome to have its own dwell measured before it may enter a cell, so the 32 seeds are not inserted on creation. They are held in a pending queue and played in sequence; each is inserted after it has been heard. Until the archive holds at least two occupied cells, CROSSOVER_RATE is treated as 0 — see the empty-archive rule in §7.3.

5.1 Priors still to be specified

The priors decide what the search has to work with. Everything else in this document is machinery for searching; §5 determines whether there is anything worth searching. It is currently specified to less depth than that leverage warrants, and the following are genuinely absent rather than deliberately left open. **An implementer must not silently choose defaults for these** — they need deciding, and the decisions belong in this section.

1. **fundamental_cents.** The harmonic prior — 65% of all pitch draws, the single most consequential prior in the document — is expressed relative to it, and its own initial distribution is nowhere given. Nor is its declared range.
2. **Envelope node priors.** Amplitude and pitch envelope node levels, times, curves and tensions have no prior at all, so they default to uniform in stored space: an average amplitude node at roughly -28 dB with enormous variance. Given that the sine-wave-speech argument (§12) puts sonic complexity in time-varying envelopes rather than oscillator count, this is the largest single gap in the section.
3. **Shape weights and switches.** Neither the four switch probabilities nor the weight distribution is given. If all four enable at $p = 0.5$ with uniform weights, most waves become a blend of all four shapes, which is a far narrower timbral palette than it appears — averaging shapes tends toward the sine-ish. A prior favouring few enabled shapes yields more distinct timbres, and the choice is consequential.
4. **gain_out mapping.** “Linear amplitude $\propto 1/k$ for the k -th active wave sorted by pitch” needs the actual dB mapping, a normalisation constant, the behaviour at $k = 1$, and whether the sort is ascending (lowest wave loudest) or descending. Ascending is a real spectral decision and is not currently stated either way.
5. **tempo_bpm.** The time prior snaps to $60/\text{tempo_bpm} \times q$ without specifying tempo’s own draw. Log-uniform over 30–300 is the natural choice, since tempo is perceptually logarithmic, but it is not stated.
6. **Joint behaviour of duration, mid_wait and pre_wait.** All three are drawn independently. A 5 ms duration with a 30 s mid-wait is one click every thirty seconds; a 25 s pre-wait in a 60 s render silences the wave for most of it. Either specify a joint prior, or state the independence deliberately and accept that a substantial fraction of waves will be inaudible in practice.
7. **phase** — presumably uniform, but say so.
8. **Unspecified global initialisations** — master_gain, sigma_global, p_switch_flip_scale, mutation_fraction.
9. **All 14 visualiser genes** — no ranges, no priors, no defaults (§11.1).

5.2 Prior sanity check — automated, before any listening

Before Gate 1a consumes anyone’s attention, render **1,000** random genomes and report the distribution of: peak amplitude, RMS, fraction of the render below -60 dBFS (silence), number of onsets, spectral centroid, and the count of genomes that clip or are effectively silent.

This is a **plumbing check, not a quality judgement**, and must be reported as raw distributions with no verdict attached. It answers questions like “are 80% of random genomes silent” or “does everything clip”, which are facts about the priors rather than claims about what will score well. It is fully automatable, costs nothing, and should run in the overnight pass so that a listening session is never spent discovering that most renders are silence.

6. VARIATION

6.0 The per-listen sequence

Everything in §6, §7 and §8 is a step in a single loop that runs once per listen. Read this before any parameter values.

1. **Select a cell, then a resident within it** — that resident is the prime parent. (§7.3)
2. **If breeding:**
 1. Select a prime parent from the archive. (§7.3a)
 2. Roll for crossover. If it fires, draw a partner from the whole archive by similarity-weighted sampling — every genome is a candidate, closer ones far likelier. (§6.4, §6.6)
 3. Assemble the child — waves inherited intact and in place from whichever parents are involved. (§6.8)
 4. Apply duplication. (§6.3)
 5. Apply mutation across the whole genome. (§6.1, §6.2)
3. **Render** the genome to SAMPLES. (§2.4)
4. **Play** it.
5. **Record dwell** at the next keypress, or at L if the render completes. (§8.3)
6. **Compute fitness** — the mean of this genome's own observations, combined with its ancestors' by relatedness weighting. (§8.2)
7. **Compute behaviour descriptors** from the samples; these give the target cell. (§7.1)
8. **Add the child to its target cell**, evicting a random resident if the cell is full. (§7.4)
9. **Update the render-length servo.** (§9)

Every branch in the loop is decided before the Creature is played. The 15% / 51% / 34% split quoted in §6.4 describes how steps 1 and 2.2 are chosen in advance; it is not a set of labels applied to a child after the listener has judged it.

No child's dwell record is thrown away. Every offspring enters its target cell (§7.4); its record is persisted (§12), is training data for the Predictor (§10), and feeds the offspring-yield statistic (§7.6). What it does not do is alter its parent's fitness.

6.1 Pipeline and terminology

Pipeline order: **select** → **recombine** → **duplicate** → **mutate**. Mutation is applied to the entire child genome after recombination and duplication, without regard to which parent each gene came from. Inherited genes are fully eligible for mutation; freezing them would confine the search to the non-inherited fraction, which shrinks as the archive converges.

Mutation-only means exactly what it says: a breeding event with one parent produces its child by mutation alone. Roughly half of all events are like this and there is no other name for them. What this specification rejects is mutation-only *as a system policy* — the position that recombination should never happen at all. That is the only distinction between the two uses of the term.

Duplication (§6.3) is a variation operator, not part of recombination. Slot-preserving inheritance governs how waves cross between parents; duplication governs how a wave is copied within a genome. They operate at different stages and do not constrain each other.

6.2 Continuous genes — self-adaptive Evolution Strategies

```
 $\sigma' = \sigma \cdot \exp(\tau' \cdot N(0,1) + \tau \cdot N_i(0,1))$   
 $\tau' = 1/\sqrt{2n} = 0.0091 \quad (n = 6102)$   
 $\tau = 1/\sqrt{2 \cdot \sqrt{n}} = 0.0801$   
 $\sigma \text{ floor } 0.002, \quad \sigma \text{ ceiling } 0.5, \quad \sigma \text{ init } 0.05$   
 $\text{gene}' = \text{gene} + \sigma' \cdot N(0,1) \quad // \text{ stored space, reflect at bounds}$ 
```

mutation_fraction sets the proportion of continuous genes receiving a draw each reproduction;
p_mutate_wave modulates it per wave.

6.2b Discrete genes

- **Binary switches:** per-class rate p_{class} , calibrated by §13.2. Base $p_0 = 0.004$, scaled by $p_{\text{switch_flip_scale}}$.
- **Routing indices:** reassign with probability 0.02, uniformly over all 64 slots; depth $\times 0.05$ on change (§4.3).
- **Node counts:** ± 1 with probability 0.03.

6.3 Duplication

A wave may be copied whole into one or more other slots of the same genome, overwriting whatever is there. This is a variation operator applied after recombination and before the per-gene mutation pass; it is **not** part of crossover and is unaffected by slot-preserving inheritance.

Trigger. With probability $p_{\text{duplicate}}$ (global gene, init 0.08) per reproduction.

Source. One wave drawn uniformly from the genome's active waves. If none are active, no duplication occurs.

Targets. Draw n_{targets} : 1 with probability 0.75, 2 with 0.20, 3 with 0.05. Draw that many distinct target slots uniformly from all 63 other slots — muted or active, without restriction. Restricting targets to muted slots would make overwriting a working wave unreachable, which invariant 2.1 forbids; the multi-slot tail exists for the same reason, since a wave propagating into several slots at once must remain reachable even though it is a large jump.

Arrival. The copy is written into the target slot with active set true, then mutated at 3σ so the copies diverge immediately rather than being bit-identical.

- **Target slot was muted:** the copy arrives at full gains. Nothing audible is destroyed and the change is purely additive.
- **Target slot was active:** the copy's gain_out is multiplied by **0.05** on arrival, and any enabled routing edge elsewhere in the genome that pointed at that slot has its depth multiplied by **0.05**.

The attenuation exists for the same reason as the reroute rule in §4.3. Overwriting an active slot is otherwise a compound change — one voice destroyed and a different one appearing at full strength in the same step. Attenuating the arrival decomposes it into a single clean structural deletion plus a quiet new wave that can grow if selection favours it.

Modulation indices inside the copy. Slot indices are absolute, so:

- If $\text{pm_source} / \text{am_source}$ pointed at a slot other than the source wave's own, the copy keeps the same index unchanged and at full depth. The copy should inherit its modulator — that is what makes it sound like the original.
- If the source wave was self-modulating ($\text{pm_source} == \text{source_slot}$), remap the copy's index to the copy's own new slot, preserving self-modulation rather than leaving the copy modulated by its original.

What this buys. Duplication-and-divergence: the copy is free to drift while the original keeps working, which is how real genomes acquire new function. Concretely here it is the only operator that reaches a chorus, a detuned pair, or a rhythmic echo of an existing voice in a single step. Point mutation cannot get there, because it would have to construct the second voice from scratch and pass through a long stretch of unrelated intermediate sounds to do it.

6.4 Recombination rate 0.50

Two independent dials. They are frequently conflated and must not be.

Three independent dials. They are frequently conflated and must not be.

Dial	What it controls	Value
CROSSOVER_RATE	how often a breeding event involves any partner at all	0.50

Dial	What it controls	Value
n_partners	how many partners take part when it does	gene, 1 - 8, init 1.4
partner_influence	how much of the genome comes from partners in total	gene, 0 - 0.5, init 0.15

Worked consequence at the initial values: half of all breeding events involve at least one partner. Of those, 60% draw one partner and 40% draw two. Whatever the count, partners collectively supply on average $0.15 \times 64 \approx 10$ of the 64 wave slots, split between them; the prime parent supplies the rest.

So CROSSOVER_RATE = 0.50 does **not** mean half the genes come from a second parent — averaged across all breeding events that figure is about 7.5% of slots.

There is no failure case. Partner selection (§6.6) samples from the whole archive with a kernel that never reaches zero, so a partner is always returned. No threshold, no retry loop, no fallback to a single parent. The realised recombination rate equals CROSSOVER_RATE exactly.

Per listen: **50% single-parent breeding events, 50% multi-parent breeding events.** Most individual events are still mutation-only; the system is not (§6.1).

The linkage argument, correctly framed. Crossover cutting between co-adapted genes is real, and co-adaptation in this genome is severe — a wave and the waves it modulates form a functional unit. But the standard reason to fear this comes from generational algorithms, where a disruptive recombinant enters the next generation and propagates. That is not the situation here. Under a deep-grid archive a disruptive child occupies at most one of D slots in one cell and is randomly evicted in time; it cannot damage the population.

The real cost of a disruptive recombination is therefore a wasted listen (§2.3b), not harm. That is a bounded, known cost, and it means the protections are cheaper to relax than they would be in a generational setting. Two have been relaxed on exactly this reasoning — CROSSOVER_RATE raised from 0.40 to 0.50, and the partner kernel widened (§6.6).

Two protections are **kept**, because they cost nothing at all: wave-intact slot-preserving inheritance (§6.8), and the depth-attenuation repair pass. Neither spends a listen, neither removes anything from reach, and both are the difference between a recombination producing something related and producing something arbitrary.

One caveat that cuts the other way and should be known: under a deep grid a poor child does **not** simply fail to displace an incumbent. Every offspring enters its cell, and when the cell is full it evicts a uniformly random resident (§7.4). So a bad child can evict a good one by chance. The damage is bounded at $1/D$ of one cell's information and is self-repairing, since a good genotype is typically represented by several near-replicas among the residents — but “no harm, no foul” is not quite exact.

The system decides this question itself. partner_influence is self-adaptive, so if recombination is disruptive in this encoding the gene will drift toward zero, and if it helps it will drift up. Log the population mean each 100 listens. That trace is the experimental readout on linkage, and the rate above should be revised in the direction the gene moves rather than by argument.

6.5 Compatibility distance

A scalar $D(A, B)$ measuring how different two genomes are. Three terms, each normalised to $[0, 1]$, each measuring a different kind of difference.

```
d_switch = (number of the 768 binary switches whose states differ) / 768
d_active = 1 - |active_A n active_B| / |active_A u active_B|           // Jaccard distance
d_global = mean |gene_A - gene_B| over the 22 global genes, stored [0,1] space
```

$D = 0.4 \cdot d_switch + 0.4 \cdot d_active + 0.2 \cdot d_global$

What each term measures, and why it is there.

- **d_switch — structural difference.** Which waves exist, which waveshapes are enabled, which envelopes are engaged, which modulation routes are live. This is the coarse shape of the genome.
- **d_active — slot correspondence.** Whether the two genomes are using the same slots at all. This term exists separately from d_switch even though active is one of the 768 switches, because slot-preserving inheritance (§6.6) depends specifically on slot-*i* occupancy corresponding between parents, and with only ~3 slots active of 64 the active bits are 64 of 768 and would be swamped. Jaccard distance over a sparse set has the right sensitivity: two genomes using entirely different slots score 1.0 regardless of how similar everything else is.
- **d_global — tuning difference.** Fundamental, tempo, mutation meta-parameters, visualiser genes. Small weight because it carries the least structural consequence for crossover.

Continuous per-wave genes are deliberately **not** included. Two genomes with the same waves in the same slots at slightly different tunings are exactly the pair crossover handles best, and including their differences would penalise that case for no reason.

Worked intuitions. These are design expectations, to be measured empirically at Gate 2 (§13) and used to calibrate §6.6.

Pair	Expected D	Why
Parent and child, mutation only, no switch flip	~0.005	one switch flip in 768 at most, active set unchanged, globals move by $\sim\sigma$
Parent and child where the flip hit active	~0.105	Jaccard jumps to 0.25 when a 3-wave genome becomes a 4-wave one — correctly, since adding a voice is a large structural change
Two siblings	~0.01 - 0.2	roughly twice a parent-child step
Two unrelated archive elites	~0.55	switch disagreement ~0.25, disjoint active sets giving Jaccard 1.0, globals ~0.33

A property of the Jaccard term worth knowing. Adding one wave to a genome that has k active gives $d_{\text{active}} = 1/(k+1)$ exactly. So going from 3 waves to 4 scores 0.25, while going from 9 to 10 scores 0.10. The diminishing structural significance of each additional voice falls out of the metric for free, with no special-casing: the fourth voice really is a larger change to a sound than the tenth, and the metric says so without being told.

The archive-wide median distance is written D_{med} and is measured continuously from random pairs of occupied cells, refreshed every 100 listens. All selectivity in §6.6 is expressed relative to it, so no constant in this specification depends on the absolute scale of D .

6.6 Partner selection — soft, distance-decaying, never zero

Requirement: close relatives should be much more likely to combine, and **every genome must remain a possible partner with non-zero probability.**

A hard threshold cannot satisfy this. An earlier version of this specification gated crossover on $D < 0.25$ with a fallback when no partner qualified, which made every pairing beyond the threshold *unreachable* rather than merely unlikely — a violation of invariant 2.1 (§2.3, third recorded instance).

The kernel. When a crossover event fires, each partner is drawn from all occupied cells except the prime parent, with probability proportional to:

$$w(A, B) = \exp(-D(A, B) / (\lambda \cdot D_{\text{med}})) \quad \lambda = 0.25$$

Exponential rather than Gaussian: each additional $\lambda \cdot D_{\text{med}}$ of distance costs the same constant factor, which makes the parameter interpretable, and the exponential's heavier tail keeps distant pairings rare rather than numerically impossible. A Gaussian at comparable selectivity puts a partner at twice the

median distance below one in 10^7 — unreachable in practice, and therefore the same defect in a different costume.

Selectivity at $\lambda = 0.25$, in interpretable terms:

- A partner at the median distance is **1/55** as likely as an identical genome.
- **A partner at twice the median distance is 55× less likely than one at the median.**
- Each $0.25 \cdot D_{\text{med}}$ of additional distance costs a factor of e .

λ was widened from 0.18 to 0.25 on the reasoning in §6.4: the cost of a poorly-matched pairing is one wasted listen rather than damage to the population, so buying reachability with it is cheap. At 0.18 a partner at twice the median was $260\times$ less likely, which is rare to the point of being effectively unreachable within a realistic listen budget. Rare must remain genuinely reachable, not vanishingly so.

Realised behaviour. On an archive holding ~ 200 elites and a handful of close relatives of the prime parent, roughly **two crossovers in three draw a close relative**, and the remaining third reaches across the archive. The old hard threshold made those figures 100% and 0%. Distant pairings are uncommon, do happen, and are the only events that can combine two independently discovered structures.

Procedure. Compute D against every occupied cell and sample proportional to w , without replacement when more than one partner is required (§6.8). This replaces the previous sample-20-and-give-up loop and is strictly better: no failure case, no fallback path, and the realised crossover rate equals `CROSSOVER_RATE` exactly. Cost is at most 255 distance computations of ~ 850 operations each per crossover event — negligible.

Diagnostic. Log the mean D of accepted partners against D_{med} . If the two converge, the archive has speciated to the point where prime parents have no close relatives left, and λ should be revisited.

Similarity-weighted mating is the same mechanism as speciation in NEAT, softened so that no pairing is forbidden.

6.7 Genotypic, not genealogical

Two things could be meant by “close relatives”:

- **Genotypic similarity** — the genomes are actually alike, whatever their history. This is what D measures.
- **Genealogical relatedness** — they share recent ancestry, whether or not they still resemble each other.

Decision: genotypic, and genealogy is not used at all.

Genotypic is primary because what mechanically determines whether crossover destroys structure is whether the genomes are similar *now*. Shared ancestry is only a proxy for that, and under an active mutation regime a decaying one.

A genealogical bonus term was specified and has been **removed**. The argument for it was that slot-preserving inheritance assumes the parents’ slot- i waves are *homologous* — descended from a common ancestral wave — and that genotypic similarity is evidence of homology rather than proof, since two genomes could hold similar slot-3 waves by convergence instead of descent.

That argument is sound and empirically irrelevant here. In a 6,102-parameter space, convergence is not a realistic route to similarity: two genomes do not arrive at matching waves in matching slots by coincidence. Genotypic closeness therefore already implies common descent, and the bonus buys nothing while adding a kernel term, a constant, and a coupling between the recombination machinery and the fitness-provenance machinery.

Revisit if the encoding changes. The argument turns entirely on dimensionality. If an indirect encoding is ever adopted (§14), the genome shrinks to a few dozen genes, convergence becomes a real possibility, and the genealogical term should be reinstated. `contrib` is maintained for §8.2 regardless, so it remains free to add.

6.8 Waves are inherited intact, in place

The crossover unit is the **whole wave** — all 95 genes together. A wave with some parameters taken from another wave is, in practice, one of the two waves plus noise, which mutation produces better and more cheaply.

Operator. Given a prime parent and k partners drawn by the kernel (§6.6), then for each of the 64 slots independently:

- with probability $1 - \text{partner_influence}$, take the wave from the **prime parent**;
- otherwise take it from one of the k partners, chosen uniformly among them.

Global genes cross uniformly per gene across all $k + 1$ parents.

So `partner_influence` is the total share of slots not coming from the prime parent, split evenly between however many partners there are. The two genes stay orthogonal: `n_partners` sets **how many** sources, `partner_influence` sets **how much** in total comes from them. Raising the partner count does not increase the amount of foreign material — it divides the same share more ways, which makes each individual partner's contribution smaller and more fragmentary.

Number of partners. `n_partners` is a global gene in $[1, 8]$, init 1.4, giving the expected count when crossover fires:

```
k = floor(n_partners)    with probability 1 - frac(n_partners)
    ceil(n_partners)     with probability  frac(n_partners)
```

At 1.4 that is one partner 60% of the time and two 40%. The range is deliberately generous rather than capped at some safe value: high partner counts are self-limiting, because a child assembled from five sources is close to a shuffle and will rarely survive its cell, so selection will push the gene back down without needing a ceiling to do it.

`n_partners` is inherited from the prime parent and is a variation gene, permitted under §2.5. Like `partner_influence` it governs the intake of *other* parents' material rather than weighting its own carrier's contribution, so it is clear of the segregation-distortion exclusion. Log its population mean alongside `partner_influence`; together they are the readout on whether multi-parent recombination helps in this encoding.

This is low-probability uniform crossover at wave granularity, deliberately much less disruptive than a 50/50 split.

Slot-preserving. A wave inherited from any parent lands in the **same slot index** it occupied in that parent. This keeps routing indices meaningful: because crossover is distance-restricted, the parents' slot- i waves are homologous, so a routing index pointing at slot i still points at something similar.

Repair pass. After assembly, for every routing index in the child pointing at a slot whose occupant came from a different parent than the referring wave did, and whose `active` or `gain_mod_on` state differs between the two parents: retain the routing index and multiply the depth by 0.05 (§4.3). The connection survives in a form that can regrow rather than being severed or arriving at full strength into an unexpected target.

Provenance. At assembly, record `src[0..63]` — which parent supplied each slot. This is required by §8.2 and costs 64 bytes.

7. SEARCH — MAP-ELITES

7.1 The archive

16 × 16 = 256 cells over two behaviour descriptors, both computed from `SAMPLES` over the full render.

Axis 1 — temporal development. Divide the render into 8 equal segments. Compute a 13-coefficient MFCC mean vector per segment. The descriptor is the mean pairwise Euclidean distance between segment vectors. One end is a static drone, the other a sound whose character changes across its length. Log-scaled, 16 bins.

Axis 2 — harmonicity. Spectral flatness (geometric mean of the power spectrum divided by its arithmetic mean), averaged over frames. One end is pitched and tonal, the other noisy. Log-scaled, 16 bins.

Criteria an axis must meet. A descriptor must be (i) computed from SAMPLES, (ii) audible — a listener can hear the difference between one end and the other, (iii) a dimension along which variety is wanted, (iv) approximately independent of fitness, so the archive does not merely rank quality along an axis.

Both axes are chosen against those four criteria and nothing else. **Neither is a prediction that sounds at any point on either axis will score better** (§2.3). A drone may be tedious or transcendent; a rapidly developing sound may be engaging or chaotic. The axes exist to guarantee variety and to make the archive legible, not to encode a guess about what wins.

Brightness was rejected on criterion (iii). Event density largely duplicates axis 1.

Descriptors falling outside the axis ranges are clamped into edge cells, never discarded (§2.1).

Axes are global constants, not genomic (§2.5).

Hashing the genome into a cell index is not an acceptable substitute for descriptors. The archive's value is locality in behaviour space: neighbouring cells hold similar solutions, so a cell is a stepping stone toward its neighbours and improvements propagate across the grid. A hash destroys that adjacency — mutations land in unrelated cells among unrelated residents, producing 256 independent random-restart hill climbers with no cross-pollination, which is strictly worse than a single population.

7.2 Deep cells

Each cell holds up to $D = 8$ residents, not a single elite. This is Deep-Grid MAP-Elites (Flageat & Cully, ALIFE 2020), and it exists to solve a specific bias in the noisy case.

The problem it solves. Under a single-elite archive, an incumbent that has been heard several times has an averaged, denoised fitness, while a challenger has been heard exactly once. Comparing them systematically favours incumbents that got lucky on an early observation, and no subsequent challenger can dislodge them because they are being compared against a lucky number rather than a true one.

How deep cells fix it without spending listens. A cell's residents are, over time, near-replications of one another — offspring are generated from archive members, so a cell accumulates variations on whatever lives there. The sub-population therefore *implicitly samples* the region, and the cell's aggregate is a denoised estimate that cost nothing beyond the listens already being spent on generation. This replaces explicit re-evaluation entirely.

Load-bearing assumption — behavioural locality. The paragraph above is only true if **offspring tend to land in their parent's cell or one adjacent to it**. The original paper states this as an assumption about the smoothness of the descriptor landscape; here it is a hard dependency and must be treated as one.

Two descriptors over 256 cells is an extremely coarse partition of a 6,102-parameter space, so the genome-to-cell map is massively many-to-one and two Creatures sharing a cell need not resemble one another at all. That in itself is tolerable — a cell is a niche, not a claim that its occupants are alike. What is *not* tolerable is offspring scattering across the grid: if a cell's residents are mutually unrelated, then random eviction is not “continually re-questioning a lineage”, it is destroying unrelated good genotypes at random, and the debiasing argument collapses into noise.

The distinction is worth holding precisely:

- **Within-cell heterogeneity** — cell-mates may sound different. Inherent to any coarse descriptor, true of plain MAP-Elites too, not fatal.
- **Behavioural locality of variation** — offspring land near their parent's cell. This is what the deep grid actually depends on. If it holds, a cell fills over time with one dominant family regardless of what it *could* hold, and implicit averaging works.

This is measured, not assumed. Gate 2b (§13.3) tests it with a pass threshold, and §13.3 lists what to do if it fails.

Newcomer protection. A resident that has just entered a cell cannot be evicted until **2 further offspring have entered that cell**. At most half a cell may be protected at once.

This exists because a newly-arrived genotype earns its slot by being selected as a parent and producing same-cell offspring, and it cannot do that if it is evicted first. The protection is **blind to fitness** — it keys on arrival order only — so it does not reintroduce the bias that fitness-based eviction would. It slows turnover slightly and that is the whole cost.

Note what this is *not*: protecting residents by **number of observations** would break the mechanism outright. In this system almost every resident has exactly one observation — genomes are heard once, replay is removed, and the cooldown (§8.5) actively suppresses repeats — so such a rule would protect nearly everyone and freeze the cell. The sampling here is implicit and lives across residents, not within them: a cell's sample size is its resident count, not any resident's listen count.

Consequence: the replay branch is removed. Earlier drafts spent 15% of listens replaying incumbents to firm up their estimates. Deep cells make that redundant, and under §2.3b those listens are better spent generating. $P_REPLAY = 0$.

Cost, stated plainly. Depth multiplies the listens needed to resolve a cell. Filling 256 cells to depth 8 takes 2,048 listens — roughly 31% of the ~6,500-listen budget for meaningful progress (§12), before any convergence within cells. $D = 8$ is chosen against that budget: deep enough that one lucky observation cannot own a cell, shallow enough that cells fill and turn over within reach. The original paper uses $D = 50$; at 256 cells that would be 12,800 individuals, more than the entire budget. Raise D if the budget grows.

7.2b The loop

On each listen:

1. **Select a cell** (§7.3), then **select a resident within it** (§7.3c). That resident is the prime parent.
2. With probability $CROSSOVER_RATE$ draw k partners by the kernel (§6.6) and recombine (§6.8); otherwise proceed with the prime parent alone. Apply duplication, then mutation.
3. Check the repeat cooldown (§8.5); regenerate if it trips.
4. Render to **SAMPLES**; compute descriptors; determine the target cell.
5. Play; record dwell (§8.3).
6. Add to the target cell (§7.4). Record the outcome against the prime parent's cell for offspring yield (§7.6).
7. Update the render-length servo (§9).

One listen, one evaluation, one archive update.

7.3 Selection

(a) Which cell. Rank-based over occupied cells, biased toward quality. A cell's fitness is the **mean fitness of its residents** — the implicit average that deep cells exist to provide.

Sort occupied cells by that mean; for the cell at rank j of N , let $r = 1 - (j - 1)/N \in (0, 1]$. Then:

```
score(j) = 0.7 · r_fitness(j) + 0.3 · r_yield(j)           // §7.6
P(cell j) ∝ score(j)^α                                     α = 2.0
```

Rank rather than raw fitness because dwell is an unnormalised duration in seconds, and any scheme proportional to raw fitness inherits every defect of roulette selection (§7.5).

At $\alpha = 2.0$ the top decile of the archive receives roughly 3.6× the selection probability of the median decile — a real bias without starving any cell, since $r > 0$ everywhere. $\alpha = 1$ is a mild rank weighting; $\alpha \geq 4$ approaches greedy and defeats the point of holding an archive. Adjust α if the archive is observed to stagnate (raise) or to collapse toward a few cells (lower).

Uniform selection over occupied cells — plain MAP-Elites — spends search effort evenly across behaviour space regardless of quality. That is a real and known property of the algorithm, and it is the reason for biasing here.

(a-edge) Empty and near-empty archive. The claim in §6.6 that partner selection always returns a partner holds only when at least two cells are occupied, since the prime parent is excluded from the draw. Until then: with zero occupied cells, play from the pending seed queue (§5); with exactly one,

CROSSOVER_RATE is treated as 0 and every event is single-parent. Restore the specified rate as soon as a second cell is occupied.

(b) What this does and does not resolve. Biased cell selection directs search effort toward the better part of the archive. It does not remove residents from poor cells: the archive still retains every occupied cell indefinitely, which is the behaviour that keeps extremes of the space represented. If it later proves desirable for the archive to shed cells, that means culling below a global fitness floor, which trades illumination for convergence and is a different algorithm. Not adopted; recorded as the available lever.

(c) Which resident within the cell. Rank-based among the cell's residents, with lexicographic parsimony breaking effective ties:

rank residents by fitness

where two residents are within 5% of each other, the lower-complexity one ranks higher

$$P(\text{resident}) \propto \text{rank_score}^\beta \quad \beta = 2.0$$

The original paper uses fitness-proportional in-cell selection; rank-based is the equivalent here, since roulette is barred system-wide (§7.5).

This is where the noise asymmetry is actually resolved. A resident with one lucky high observation is preferred, but not exclusively — less successful residents that may simply have been unlucky retain a real chance, and the container rule below evicts on a schedule that ignores fitness entirely, so a lucky one-shot cannot hold its slot indefinitely.

7.4 Container maintenance

cell = target cell from the behaviour descriptors

if |cell| < D: add the challenger

else: replace a UNIFORMLY RANDOM resident with the challenger

Every offspring enters its cell. There is no contest and no fitness comparison at insertion.

Eviction is uniformly random, and deliberately ignores both fitness and age. This is the mechanism that removes the noise bias rather than merely softening it: residents are continually re-questioned, so a slot can only be held by genotypes that keep producing offspring which land in the same cell. Stable, genuinely high-performing genotypes are selected more often, mutate into offspring that belong to the same cell, and slowly come to dominate it. Lucky one-shots are evicted at the same rate as everything else and are not replaced by their own offspring, so they wash out.

Making eviction fitness-based would reintroduce exactly the bias the depth exists to remove.

Where parsimony went. Lexicographic parsimony pressure previously lived in the insertion contest, which no longer exists. It now operates in in-cell selection (§7.3c), where it does the same job — complexity breaks ties between residents of equivalent fitness and can never override a real fitness difference — without touching the container rule. A multiplicative complexity penalty on fitness remains rejected: such penalties are hard to tune, the safe window moves as fitness scales change, too weak does nothing and too strong collapses the population to triviality.

Documented alternative, not adopted: complexity as a third archive axis would preserve simple and complex residents side by side. Rejected because a third axis multiplies the cells requiring listens to fill, and depth has already multiplied that cost by 8.

7.5 Roulette selection is barred

Fitness-proportionate selection must not be used anywhere in this system: it is sensitive to fitness scaling, degenerates when fitnesses are near-equal, and the signal here is a noisy duration in seconds.

Every genome must have its own dwell measured at least once before it enters a cell. Lineage-averaged fitness must never be the sole basis on which a genome occupies a slot.

7.6 Offspring yield

Under deep cells every offspring enters its target cell, so nothing is discarded outright. But an offspring evicts a resident at random once a cell is full, and the information in a short-lived resident's single listen

would otherwise vanish with it. It is retained as follows.

The statistic. Each **cell** carries `n_offspring` and `mean_offspring_dwell`, covering every child bred from any resident of that cell, regardless of where the child landed. Tracking this per cell rather than per resident is what makes it survive the container's random eviction: residents come and go, the cell's neighbourhood does not.

Both accumulate with a rolling window of the last 50 offspring, so a cell whose character has changed is not held to its history indefinitely.

A cell with little data has its value shrunk toward the archive-wide mean:

$$Y = (n \cdot \text{mean_offspring_dwell} + m \cdot Y_{\text{archive}}) / (n + m) \quad m = 5$$

$m = 5$ sets how many offspring a cell needs before its measured yield dominates the prior. Dwell variance is high, five observations is roughly where a mean becomes usable, and a new cell therefore sits at the archive average rather than at an extreme.

Where it is used, and only there. Parent selection (§7.3a) ranks cells on a blend of the two signals:

$$\text{score} = 0.7 \cdot r_{\text{fitness}} + 0.3 \cdot r_{\text{yield}} \quad P(\text{cell}) \propto \text{score}^\alpha$$

where `r_yield` is the cell's rank on `Y`, computed exactly as `r_fitness` is. The 0.3 weight is enough to move a cell with consistently strong offspring up roughly a decile band, and small enough that a cell the listener genuinely likes is not demoted for a run of unlucky children.

Firewall. `offspring_yield` must never enter $F(g)$, never influence container maintenance, never enter the Predictor's training target, and never appear in a diagnostic. It affects where search effort is spent and nothing else.

Why this is not a smuggled prediction (§2.3). Two alternatives were considered and one rejected:

- *Feeding a child's dwell into its parent's fitness* is **rejected**. It would change what a genome's fitness means — from “how long people listened to this Creature” to “how long people listened to this Creature and things near it” — and would make stored fitness partly a function of how many children a genome happened to have, which is selection history rather than sound. Fitness is an instrument and must stay clean.
- *Biasing cell selection by offspring yield* is **adopted**, because selection is not an instrument. It changes where effort goes, not what is measured, and it is already fitness-biased by explicit decision ($\alpha = 2.0$).

The assumption it rests on is that a parent whose children scored well is worth exploring further — that is, that the landscape is locally correlated. That is not a claim about *which sounds win*; it is a claim about the structure of the search, and it is the same assumption the locality criterion in §13.2 already makes and explicitly tests. If it fails, Gate 2 fails and the project stops there regardless.

8. FITNESS AND MEASUREMENT INTEGRITY

8.1 What fitness is

Dwell time in seconds. Nothing else. The system is agnostic as to *why* a listener kept listening.

No claim is made anywhere in this specification about which sounds will produce long dwell. That is what the system is for finding out (§2.3).

8.2 Relatedness-weighted lineage averaging

A single listen is one observation of a high-variance quantity. Fitness therefore averages over a genome's own dwell and its ancestors', which borrows statistical strength from genotypic neighbours — valid because §13.2 establishes that they are phenotypic neighbours.

Under multi-parent recombination a flat depth weighting is wrong: an ancestor that contributed 5% of the current genome would count as much as one that contributed 60%. Each ancestor's dwell is therefore

weighted by **the fraction of the current genome actually inherited from it**. This is relatedness weighting in Hamilton’s sense — the coefficient of relatedness between the genome and each ancestor.

Provenance tracking. Each genome stores contrib, a map from ancestor id to inherited fraction, truncated to depth 3 and to the 8 largest contributors.

At birth, from the src[0..63] array recorded in §6.8:

```
direct(p)      = |{ i : src[i] == p }| / 64
contrib_child  = { p : direct(p) } ∪ { a : Σ_p direct(p) · contrib_p(a) }
```

Global genes are ignored in this calculation; they are 22 of 6,102 parameters and folding them in changes nothing at three significant figures.

In a single-parent event direct(parent) = 1.0, so every ancestor’s contribution is 1.0 and the scheme reduces exactly to flat depth weighting. It is a strict generalisation, active only where recombination has actually mixed lineages.

Multiple observations of one genome. A genome heard more than once — through B, or because an identical genome was independently generated — accumulates observations. dwell(g) is the **plain arithmetic mean of that genome’s own attended observations**, with unattended listens entering at weight 0.25.

Relatedness weighting operates strictly *between* individuals and never within one. Each dwell term in the formula below is itself a plain mean of that individual’s own listens; the weights combine individuals, not observations.

Fitness:

```
w_0      = 0.5
w(a)     = depth_weight(a) · contrib(a)      depth_weight = 0.3 at depth 1, 0.2 at depth 2
F(g)     = [ w_0 · dwell(g) + Σ_a w(a) · dwell(a) ] / [ w_0 + Σ_a w(a) ]
```

Lineage depth is a **global constant** of 3 (§2.5).

8.3 Dwell measurement

Dwell is the time from playback start to the next keypress, subject to the following. All parameters are global (§2.5).

Mechanism	Behaviour
Page Visibility API Window focus Input idle	Pause the dwell clock while document.hidden Pause the dwell clock on blur, resume on focus If no keyboard, pointer or touch event for 90 s , mark the listen unattended; recorded dwell is the time to the last input plus 90 s
Render completion	When the render reaches its end, dwell is recorded at exactly L, flagged completed, and the clock stops. Time spent after the render ends does not accrue.
Minimum listen	Dwell below 0.35 s is a double-tap; discard the record entirely

Dwell is therefore bounded to [0.35, L], and is **right-censored at exactly L**. That clean censoring point is what the servo in §9 depends on; no separate absolute cap is used.

Listens flagged unattended enter the record with weight **0.25** in the lineage average rather than being discarded — discarding them would destroy the signal that something held attention for a long time.

There is no looping and no auto-advance. Looping would make repeated material indistinguishable from developing material; auto-advance would destroy the skip signal. When the render ends the sound stops, the visuals hold, and the system waits for a keypress.

8.4 Diagnostics are for measurement faults only

Log the full dwell distribution, the proportion of completed and unattended listens, and the servo's history. These exist to detect **faults in the instrument** — the inattention confound, whether the servo is tracking, how much of the signal is being censored. They are never a verdict on whether the population is doing well. No shape of dwell distribution is a sign of health or of trouble in the Creatures; the system does not know what a good population looks like, and any diagnostic claiming to is a smuggled prediction (§2.3).

8.5 Repeat cooldown

Listens are the scarce resource (§2.3b), and repeated exposure spends them badly: a listener hearing the same Creature several times in short succession is measuring their own novelty exhaustion rather than the sound. A second dwell recorded minutes after the first is not a second sample of the same quantity.

Constraint. No genome may be played twice within a rolling window of $W = 30$ listens. Keep a FIFO of the last 30 genome hashes; if a generated candidate's hash is present, discard and regenerate, up to **5 attempts**, then accept regardless.

At a mean dwell of around 15 s, thirty listens is roughly eight minutes. That is the horizon over which a repeat is more likely to be measuring boredom than the Creature.

This is a soft constraint, not a rule about what may exist. It never prevents a genome from being generated, held, selected or heard — only from being heard *again too soon*. Under invariant 2.1 that distinction matters: nothing is unreachable, only rescheduled.

Exemption: the B key. A listener choosing to return to a parent is an explicit request, not a system-imposed repeat, and is never blocked. Dwell from a B replay is recorded normally and averaged into that genome's observations (§8.2).

An earlier framing of this idea was the rule “you never hear the same sound twice”, stated as a hard property of the system. It is not a hard rule and never needed to be; the operative version is the cooldown above.

8.6 Annotations — the feedback field

A text field in the interface for remarks written as they occur. On submit the text is written to the log with a timestamp and the field clears.

Hard rule, stated here because this is exactly the feature a later agent would helpfully wire into the search: annotations never enter the search. Ever.

They must not enter $F(g)$, the behaviour descriptors, the Predictor's inputs or target, any selection decision, container maintenance, the servo, or any gate threshold. They live in their own notes object store with no join path into the fitness pipeline.

The reason is the mission statement, which rejects stated preference and takes measured engagement only. Free text is stated preference in its purest form. The annotation log exists for a human reading it later — the project owner, or a cold evaluator interpreting the run — and for nothing else. The evaluation protocol (§15) may read notes as *context* for interpreting the logs, and may not derive from them any recommendation to steer the search.

Anchoring. A remark detached from what prompted it is close to useless, so each note records:

`note_id`, `timestamp_ms`, `session_id`, `listener_id`, `listen_id`, `genome_id`, `cell_x`, `cell_y`, `dwell_at_note_s` (dwell accrued before the field took focus), `render_position_s` (playhead position at that moment), `L_at_note`, `time_composing_ms`, `text`.

`time_composing_ms` is worth having: a long note means a long pause, and an evaluator should be able to see that without inference.

Interaction, and the two contamination problems it creates.

Problem one: typing accrues dwell the Creature has not earned. This is the inattention confound arriving through the front door — the listener is attending to a keyboard, not to a sound.

Problem two: SPACE types a space instead of skipping while the field holds focus, as do M and B.

Both resolved by the same mechanism:

Event	Behaviour
F, or clicking the field	Field takes focus. The dwell clock pauses , by exactly the mechanism used for visibility and blur (§8.3). Audio continues playing.
While focused	All transport keys are inert as transport — they type. Do not fight the browser on this; disabling them would be worse.
Enter	Submit. Note written immediately. Field clears. Focus returns to the body, so transport keys work again. Dwell clock resumes.
Shift+Enter	Newline, stay focused.
Escape	Discard. Field clears, focus returns, clock resumes.

Audio keeps playing during composition deliberately: stopping it would mean writing about something no longer audible. If the render completes while the field has focus, record completed at L as normal.

Flagging. A listen during which a note was composed is flagged annotated and carries `n_notes`. Unlike unattended, it enters the lineage average at **full weight**, because the clock was paused and the recorded dwell is therefore honest. The flag exists so an evaluator can check whether annotated listens behave differently from the rest — if they do, the pause mechanism is not working.

Writes are immediate. Do not batch notes or hold them until the listen ends. The point of the feature is capture at the moment of noticing.

9. RENDER LENGTH SERVO

Render length L is a global system parameter, not a gene. A genome that rendered longer would have more dwell available to it, making length a gene that touches the fitness function (§2.5).

9.1 The censoring asymmetry, which determines the design

Dwell observations recorded at length L are **right-censored at L**: a listen flagged completed tells you the listener would have stayed at least that long, not how much longer.

The consequence is asymmetric and it is why the algorithm below is shaped as it is:

- **Shorter lengths can be computed retrospectively.** For any $L' < L$, every recorded listen either ended before L' — in which case its dwell is exactly what it would have been at L' — or reached L' , in which case it would have maxed out at L' . The proportion that would have maxed out at any $L' < L$ is therefore known exactly from data already held.
- **Longer lengths cannot.** No observation carries information about behaviour beyond L.

So shrinking is **computed** and extending is **triggered**. Extension is a step into the unknown and must be re-measured; shrinking is arithmetic on data in hand.

9.2 The algorithm

After every listen, over the trailing window of the last X listens:

Let T = max acceptable proportion of listens that max out
Let p = proportion of the window flagged 'completed'

if $p > T$:

```

L ← min(L · 1.5, L_CEILING)           // extend; blind above L, so step and re-measure
else:
    k  = floor(T · X)
    L' = the (k+1)-th largest dwell in the window           // shortest L with p(L') ≤ T
    L  ← max((L' + L) / 2, L_FLOOR)                       // move halfway

```

L' is an order statistic: setting the length just above the (k+1)-th largest dwell leaves exactly k listens at or above it, so the maxing-out proportion is $k/X \leq T$ and no shorter length satisfies the constraint.

The halfway move damps the shrink against window noise. The computed value L' is correct for the window observed but the window is a sample; moving halfway concedes the direction without committing to the magnitude.

Guards. Do not run the servo until the window is full. Apply a change only if $|L_{\text{new}} - L| / L > 0.05$, to prevent thrashing. Log every change with the window statistics.

Because primary fitness is absolute seconds, stored fitness values remain comparable across a change in L.

9.3 Values

Parameter	Value	Reasoning
X	100	With $T = 0.10$, $k = 10$, so L' reads the 11th-largest of 100 — a stably estimated quantile. Small enough that L tracks a changing population within roughly a session's data.
T	0.10	At 10%, the ceiling is rarely the binding constraint but is still occasionally informative. Much lower and L grows without bound chasing rare long listens; much higher and the top of the fitness signal is routinely censored.
Extension step	x 1.5	Geometric, so scale-free. Reaches the ceiling from 60 s in a handful of adjustments without large overshoot into unmeasured territory.
L_FLOOR	15 s	Below roughly this, a render cannot express the pre-wait / duration / mid-wait structure the genome describes.
L_CEILING	300 s	Engineering limit, not a design one: 300 s of mono float32 at 44.1 kHz is ~53 MB in a single AudioBuffer. Raising it requires chunked or streaming synthesis rather than one buffer.

9.4 No special cases

Earlier drafts raised a diagnostic when early skips dominated, on the theory that this indicated a fault in the priors. That has been removed. A run of early skips may simply mean the recent population is

unappealing; it demonstrates nothing about the priors, and treating it as a fault would be a claim about what ought to score well (§2.3).

The servo handles the situation without a special case: many early skips means few listens maxing out, so the shrink branch computes a shorter L and stops rendering audio nobody hears. That it needs no special case is a point in the design’s favour.

10. THE PREDICTOR

Operates on GENOME only and therefore performs no renders.

10.1 Failure mode

Surrogate-assisted evolutionary algorithms are vulnerable to **false optima** — points that optimise the model but not the true objective. Given hundreds of thousands of free evaluations, the search will find the model’s blind spots and occupy them. Every parameter below exists to contain this.

10.2 Specification

Parameter	Value
Architecture	ensemble of 5 MLPs, 3 hidden layers × 256 units, ReLU, dropout 0.1
Inputs	full genome (6,102, stored space) + hour-of-day (sin/cos) + listener id (one-hot) + session index + position in session + current L
Head A	log dwell time
Head B	behaviour descriptor (development, harmonicity)
Minimum data before first use	2,000 attended listens
Retraining	from scratch on all data, at every checkpoint
Autonomous iterations per checkpoint	1,000
Fitness used autonomously	$\text{mean}(\text{ensemble}) - 1.0 \times \text{std}(\text{ensemble})$
Trust region	reject candidates whose predicted descriptor falls in a cell with no human-evaluated sample in it or in its 8 neighbours
Re-grounding per checkpoint	60 listens: the 30 highest-predicted and the 30 highest-uncertainty archive entries
Health metric	Spearman ρ between predicted and actual dwell over those 60
$\rho \geq 0.40$	healthy; autonomous iterations may rise to 2,000
$0.20 \leq \rho < 0.40$	halve autonomous iterations to 500; raise the uncertainty coefficient to 2.0
$\rho < 0.20$	autonomous operation disabled until ρ recovers

current L is an input because dwell is censored at L and the model must be able to account for that when trained across periods with different lengths.

The mean – $k \cdot \text{std}$ rule is a Lower Confidence Bound acquisition function: it makes the search actively distrust regions where the ensemble disagrees, which is where false optima are found.

The trust region restricts the **surrogate’s** search only. It places no restriction on what a listener-driven search can reach, and therefore does not violate §2.1.

10.3 Expected return

30 checkpoints × 60 re-grounding listens = 1,800 listens supporting 30,000 archive iterations, against 1,800 iterations for the same listens unassisted. Roughly 17× on listener time when healthy.

11. VISUALISER

Two input channels, both required.

Slow channel — GENOME. Delivers family resemblance. Recomputed once per Creature.

- One visual field per active wave. Position from slot index; base hue from pitch_master; size from gain_out; field style from whether the wave is a carrier, a modulator, or both.
- Modulation edges drawn as arcs between fields; thickness from depth; solid for PM, dashed for AM.
- Global palette, symmetry and motion character from the 14 visualiser genes: hue_base, hue_spread, saturation, luminance_floor, ripple_gain, bloom_radius, motion_damping, particle_density, edge_softness, pitch_to_hue_weight, amp_to_scale_weight, symmetry_order, field_blend_mode, background_drift.

Two genomes sharing most waves produce most of the same fields, so relatedness is visible.

Fast channel — SAMPLES. Delivers millisecond-to-millisecond linkage and within-render timbral change, neither of which is derivable from the genome alone.

- During synthesis, cache a **60 Hz per-wave amplitude envelope** alongside the output buffer. Free at render time, and gives exact per-wave attribution that an FFT of the mixed output cannot recover.
- Each field's instantaneous brightness and scale are driven by its own wave's cached envelope.
- Global luminance follows short-window RMS of the mix.
- A short-window spectral centroid drives a global hue offset, so timbral change within a render is visible.
- Onset detection on the mix triggers ripples.

Do not implement the fast channel as a general FFT visualiser over the mixed output. Per-wave attribution is the point.

The visualiser is part of the fitness instrument. Dwell is measured with visuals present, so what the system optimises is *attending* time rather than listening time strictly. The 14 visualiser genes are therefore under selection like any others, through their effect on dwell. This is a consequence of the design rather than a flaw in it — the genes were put in the genome deliberately — but it should be stated rather than left implicit, because it means a Creature can in principle be held onto for how it looks.

11.1 What is still undesigned

This section is specified to noticeably less depth than the rest of the document, and an implementer would have to invent most of it. That is disclosed rather than papered over. It is **not on the critical path**: every automated gate (§13) is audio-only, so the visualiser can be built last within Stage 1 without blocking anything.

The following are aesthetic decisions that belong to the project owner, not to an implementer or to this document:

- **What a “field” is geometrically.** The stated instinct is “big soft screen-filling waves of colour with ripple accents”. That has never been converted into a shape, a gradient profile, or a compositing model.
- **Layout.** “Position from slot index” — 64 slots onto a 2D canvas by what rule? Ring, grid, pitch-mapped height, force layout?
- **Ranges, mappings and defaults for all 14 visualiser genes**, and their initialisation priors (§5.1 item 9).
- **Behaviour between Creatures** — the visual counterpart of the 8 ms audio fade.
- **Onset detection method** for the ripple trigger.

Minimum viable fallback, so the harness is usable before those decisions are made: one radial gradient blob per active wave, positioned on a ring by slot index, hue from `pitch_master`, radius from `gain_out`, brightness driven per-frame by that wave's cached 60 Hz envelope; straight lines between blobs for modulation edges; global luminance from mix RMS; crossfade of 250 ms between Creatures. Deliberately plain, and to be replaced.

House conventions that must carry over regardless. Honour prefers-reduced-motion by slowing motion to 0.1x, as every Sound Sandbox visualiser does. No high-contrast luminance alternation above 3 Hz covering more than 25% of the screen. Neither is an aesthetic choice.

12. IMPLEMENTATION

Single HTML file. Vanilla JavaScript, Web Audio API, canvas 2D. No build step, no dependencies, no framework.

Audio is rendered offline into an `AudioBuffer` and played through an `AudioBufferSourceNode`. No `AudioWorklet` and no realtime synthesis constraint. Stop is an 8 ms linear ramp to zero followed by `stop()`.

Rendering at 44,100 Hz mono. Descriptor computation may downsample to 22,050 Hz.

State is persisted to IndexedDB, not localStorage. A full genome is 6,102 floats — roughly 24 KB raw — and a run of several thousand listens will exceed the ~5 MB `localStorage` quota within the first few hundred. IndexedDB quotas are typically a large fraction of free disk. Storage schema and volume management are specified in §14.

Records of evicted residents are persisted on the same terms as current ones. They are Predictor training data and they feed §7.6.

12.1 The legibility display

Not decoration. This is the primary debugging surface and must exist from the first working build.

- One lane per **active** wave, ordered by slot index; a compact strip indicating how many slots are muted.
 - Per lane: the gate pattern (pre-wait, duration, mid-wait) across the full render; height from `gain_out`; hue from `pitch_master`; distinct border style for modulator-only, carrier-only and both.
 - Arcs between lanes for every enabled modulation edge; thickness from `depth`.
 - The rendered waveform, full width, with playhead.
 - Readouts: expressed parameter count, complexity (not-kill switch count), archive cell coordinates, lineage depth, current L, live dwell timer.
-

13. BUILD ORDER AND GATES

Each stage has an acceptance test. Do not proceed past a failed gate by adding machinery downstream of it.

Stage 1 — Generator

Genome structure, priors, synthesis including the modulation matrix and cycle handling, SPACE / M / B, lineage stack, dwell logging with attention gating, legibility display.

Gate 1a. Draw 100 random genomes and audition them. At least 10 hold a listener past 10 seconds. This is a check that the generator produces something other than uniform noise, not a judgement about what should score.

Gate 1b. Take a genome that passes 1a; apply M twenty times in succession. Offspring are recognisably related to their parents. B returns reliably to the previous state at arbitrary depth.

Stage 2 — Locality calibration (§13.2)

Gate 2a. Continuous-gene locality criterion met (§13.2). Switch jump sizes measured and per-class flip rates set. Compatibility-distance distribution (§6.5) measured over random genome pairs and the worked intuitions confirmed to within a factor of two, since λ in §6.6 is calibrated against D_{med} .

Gate 2b — behavioural locality (§13.3). The deep-cell archive depends on it. Must pass before Stage 3 is built.

Stage 3 — Archive

Behaviour descriptors, deep-cell archive with random-eviction maintenance, rank-based cell and in-cell selection with lexicographic parsimony, offspring yield, similarity-weighted multi-parent crossover, duplication, provenance tracking, repeat cooldown, render-length servo.

Gate 3. After 1,000 listens: at least 40 cells occupied, at least 15 of them holding 3 or more residents, and the mean cell fitness above that of the first 200 listens.

Stage 4 — Predictor

Only after 2,000 attended listens exist.

Gate 4. Spearman $\rho \geq 0.40$ on a held-out set before any autonomous iteration is run.

13.2 The locality test

Locality — the property that genotypic neighbours map to phenotypic neighbours — is a measurable property of the representation and the precondition for evolution working at all.

Procedure.

1. Draw 100 genomes. For each, generate 20 mutants at each of $\epsilon \in \{0.001, 0.01, 0.1, 0.5\}$, with all σ forced to ϵ and switches held fixed.
2. Render 4 s of SAMPLES for parent and each mutant.
3. Perceptual distance = mean Euclidean distance between MFCC frame sequences (13 coefficients, 1024-sample window, 512 hop).
4. Compute U = median perceptual distance between unrelated random genomes. This is the ceiling.
5. Plot median distance against ϵ on log-log axes.

Continuous-gene criterion. At $\epsilon = 0.01$, the 90th-percentile perceptual distance must be **below $0.20 \cdot U$** . Repeat per gene class to locate any class that fails.

Switches are measured separately and are expected to fail the above. Non-locality is their function: they make discrete moves across fitness valleys that a continuous walk cannot cross. Excluding them from the test loses information; holding them to the continuous criterion would be wrong.

For each switch class, measure J_{class} = median perceptual distance of a single flip, as a fraction of U . Set the flip rate:

$$p_{\text{class}} = 0.004 \cdot \min(1, 0.25 / J_{\text{class}})$$

A switch class whose typical flip moves the sound a quarter of the way to an unrelated genome keeps the base rate; one that moves it the whole way gets a quarter of it. active is expected to have the largest J and to settle near $p \approx 0.001$. Publish the J table as a build artefact — it is a direct readout of the encoding's structure.

Target overall: approximately **1.0 switch flip per reproduction**, summed across all 768 switches.

Encodings chosen for locality:

Quantity	Encoding	Reason
all frequencies	log (cents)	equal genetic steps, equal perceptual steps

Quantity	Encoding	Reason
durations and waits	log	as above
output gain, envelope levels	dB	linear amplitude has almost no resolution near silence
modulation depth	log	index 0→1 is a large timbral move, 8→9 is not
envelope node times	proportions, sum-normalised	mutation redistributes rather than overflows
envelope curve	one continuous parameter	no redundancy cliff
routing indices	discrete, with depth attenuation	§4.3

13.3 The behavioural locality test

Tests the assumption the deep-cell archive rests on (§7.2): that offspring land in or near their parent's cell.

Procedure.

1. Take 200 genomes. Before the archive exists, draw them from the priors; afterwards, sample them across occupied cells.
2. For each, generate 20 offspring through the real variation pipeline. Run it twice: once with mutation and duplication only, once including crossover, and report both — crossover is expected to be less local and it is useful to know by how much.
3. Render each parent and offspring, compute descriptors, find target cells.
4. Report:
 - p_{same} — fraction of offspring landing in the parent's own cell.
 - p_{near} — fraction landing in the parent's cell or one of its 8 neighbours.
 - H_{cell} — mean MFCC distance between residents of the same cell, as a fraction of U (§13.2). A diagnostic of how heterogeneous cells actually are.

Pass threshold: $p_{\text{same}} \geq 0.35$ and $p_{\text{near}} \geq 0.70$ on the mutation-only run.

At $p_{\text{same}} = 0.35$, roughly a third of a lineage's offspring return to its cell, so across the ~8 offspring needed to fill a cell the dominant lineage gets repeated representation and implicit averaging works. Below about 0.20 a cell is mostly strangers and random eviction is noise rather than re-questioning. $p_{\text{near}} \geq 0.70$ matters separately: the stepping-stone argument for the archive (§7.1) requires improvements to propagate to neighbouring cells.

If it fails. H_{cell} tells you which regime you are in and therefore which fix applies.

Fix	When it applies	Cost
Coarser grid ($8 \times 8 = 64$ cells)	p_{same} low because cells are small relative to a mutation's descriptor step	Less diversity resolution. Also <i>cheaper</i> : $64 \times 8 = 512$ listens to fill rather than 2,048
Finer grid	H_{cell} near U — cells are enormous and heterogeneous, and the descriptor has unused resolution	More cells to fill
More descriptor dimensions	Cell-mates genuinely unlike along an axis not being measured	Cells multiply; likely unaffordable at this budget

Fix	When it applies	Cost
Descriptors chosen for mutational stability	General under-performance on p_same	None. The most principled fix — for each candidate descriptor, measure the median absolute change under an ϵ -mutation normalised by axis range, and prefer descriptors that minimise it while still meeting the four criteria in §7.1
Switch to adaptive-sampling MAP-Elites (Justesen et al., 2019)	Behavioural locality genuinely absent and not fixable by grid or descriptor choice	Single elite per cell; a challenger must beat the elite after being sampled the same number of times, and the elite is re-sampled whenever it survives. Spends listens on explicit re-evaluation — exactly what the deep grid was chosen to avoid — but depends on no locality assumption at all

The trade is clean and should be decided by the gate rather than by argument: the deep grid is cheap and assumes behavioural locality; adaptive sampling is expensive and assumes nothing.

14. LOGGING

Instrumentation must be built with each stage, not requested afterwards. What is recorded is derived from the gates (§13), the diagnostics (§8.4), and the invariants (§2) — not from what looks interesting during the build. A run that produces hours of use and logs that cannot answer the gate questions has to be re-run.

Format. One JSON object per event, appended to a JSON Lines stream in IndexedDB, plus a “download session log” control producing a .jsonl file. Separate object stores for listens, archive snapshots, servo events, and anomalies.

Genome storage. Storing 6,102 floats per listen is roughly 24 KB and will not fit for a long run. Each listen record stores a genome_id (content hash) and a **delta against its prime parent** — the indices and values of every gene that differs. Every 100th genome is stored in full as a resync point. Genomes must be exactly reconstructible from the log alone; an implementation that cannot rebuild a genome from its log has failed this section.

14.1 Per listen

Written on every listen without exception, including discarded double-taps.

Identity — listen_id (monotonic), timestamp_ms, session_id, listener_id, genome_id.

Genome — genome_delta (or genome_full on resync), expressed_parameter_count, complexity (not-kill switch count), active_wave_count, modulation_edge_count, has_feedback_cycle.

Provenance — prime_parent_id, partner_ids[], k_partners, crossover_fired, src[0..63], contrib map, duplication_fired, duplication_targets[], duplication_hit_active_slot.

Variation actually applied — not the gene values, what was *done*: n_continuous_genes_mutated, sigma_mean/min/max, n_switch_flips and which classes, n_reroutes, n_node_count_changes. Gene values as used: mutation_fraction, n_partners, partner_influence, p_switch_flip_scale, p_duplicate, sigma_global.

Partner selection — `D_to_each_partner[]`, `D_med_at_selection`, `n_cells_occupied_at_selection`.

Parent selection — `parent_cell`, `parent_cell_rank_fitness`, `parent_cell_rank_yield`, `parent_score`, `parent_cell_resident_count`, `in_cell_rank_of_chosen`.

Cooldown — `cooldown_collisions`, `accepted_despite_collision`.

Descriptors — `development_raw`, `harmonicity_raw`, `cell_x`, `cell_y`, `clamped_to_edge`.

Render — `L_at_listen`, `render_wall_ms`, `sample_peak`, `clipped`, `render_error` (nullable).

Dwell — `dwell_s`, `completed`, `unattended`, `discarded_short`, `visibility_pause_count`, `visibility_paused_ms`, `focus_pause_count`, `focus_paused_ms`, `idle_triggered`.

Fitness — `own_dwell_mean`, `own_n_observations`, `lineage_F`, and the ancestors used with their contrib, `depth_weight`, final weight and dwell.

Archive action — `target_cell`, `cell_size_before`, `inserted`, `evicted_genome_id` (nullable), `evicted_had_n_observations`, `n_protected_in_cell`, `eviction_blocked_by_protection`.

14.2 Archive snapshot — every 100 listens

`cells_occupied`, coverage (occupied / 256), a **histogram of cell depths** (how many cells hold 1...8 residents), and the sum of cell mean fitnesses as a QD-score equivalent.

Per occupied cell: `mean_fitness`, `resident_count`, `n_offspring`, `mean_offspring_dwell`, `Y_shrunk`, `resident_ids[]`.

Population statistics over all residents: mean and full quantiles of every global gene — **`n_partners` and `partner_influence` especially, since their drift is the experimental readout on linkage (§6.4)** — plus `sigma_global`, `mutation_fraction`, `p_duplicate`, `expressed_parameter_count`, `complexity`, `active_wave_count`.

Also `D_med`, and the mean `D` of partners accepted since the last snapshot.

14.3 Servo events

On every evaluation of the servo, whether or not it acts: `old_L`, `new_L` (or `no_change`), `direction`, `p_completed`, `k`, `L_prime_computed`, `window_n`, the **full sorted dwell vector for the window**, and `suppressed_by_5pct_guard`.

The sorted dwell vector matters: `L'` is an order statistic, and without the vector a later reader cannot check whether the servo computed it correctly.

14.4 Gate artefacts

Gates 2a and 2b produce tables, not runtime logs, and are written once per run as separate files: the locality curve (median perceptual distance against ϵ , per gene class), the `J_class` table, the compatibility-distance distribution with the §6.5 worked intuitions beside the measured values, and the behavioural-locality results (`p_same`, `p_near`, `H_cell`, mutation-only and with-crossover).

14.5 Annotations

The notes object store, schema in §8.6. Exported alongside everything else and readable by an evaluator, with the §8.6 firewall restated wherever the export is documented.

14.6 Anomalies

A separate stream: render failures, NaN or infinity in synthesis, clipping, cycle-detection results (`n_back_edges`), storage-quota events, and any case where a guard fired (cooldown exhausted its 5 attempts, protection blocked every candidate for eviction, partner sampling returned the prime parent).

14.7 What the logs must be able to answer

The test of this specification is whether the logs alone can answer these without access to the running system. If any cannot be answered, the instrumentation is incomplete.

1. Did behavioural locality hold in practice, over the whole run rather than at the gate? (parent_cell vs target_cell on every listen.)
 2. Did partner_influence and n_partners drift up or down, and by how much?
 3. How much of the dwell signal was censored at L, and how much was flagged unattended?
 4. Did the archive fill, and did mean cell fitness rise?
 5. How often did a resident get evicted before ever being selected as a parent?
 6. Is the encoding's expressed complexity growing, and at what rate?
 7. What fraction of listens were spent on Creatures whose parents came from the top decile of the archive?
 8. Did annotated listens differ measurably in dwell from unannotated ones? (If so, the pause in §8.6 is not working.)
-

15. EVALUATION PROTOCOL

15.1 A fresh reader evaluates, not the builder

Evaluation is performed by a session that did not produce the build. An agent assessing its own output is unreliable in a specific and predictable way: it knows what it intended, so it reads the logs for confirmation and explains away what does not fit.

The cold evaluator is given: this specification, the project entity in the vault, the log files, and the gate artefacts. Nothing else — in particular, not the build session's own account of what it did.

15.2 What the evaluation may conclude, and what it may not

This is the guard, and it exists because the failure mode has already occurred three times (§2.3).

"Evaluate how this did against the stated aims" is precisely the prompt under which a designer assumption re-enters the system. The aims are **structural**. They are not aesthetic.

In scope — the evaluator may and should answer:

- Does the generator produce anything at all worth a second listen, or is generation-zero output uniform noise? (Gate 1a.)
- Is variation local — genotypically (§13.2) and behaviourally (§13.3)?
- Does the archive fill, and does mean cell fitness rise against the opening baseline?
- Is the fitness signal usable — what is its variance, how much is censored, how much is contaminated by inattention?
- Are the mechanisms firing at their designed rates: crossover, duplication, switch flips, reroutes, distant pairings?
- Did any invariant get violated in implementation?
- Are any specified constants obviously mis-set, on the evidence of the logs?

Out of scope — the evaluator must not conclude:

- That any *kind of sound* is or is not working. Not "the drones are underperforming", not "the system is converging on noise and needs a constraint", not "the developing sounds are doing better so we should favour them."
- That the population is healthy or unhealthy on the basis of the *shape* of the dwell distribution (§8.4).
- That the output is or is not good music.

The system does not know what will score well, and neither does the evaluator. A finding that one region of behaviour space has higher dwell than another is **data, not a recommendation**: it may be reported, and it may not be turned into a proposal to bias the search toward that region. That proposal is the failure mode wearing an evidence costume.

15.3 The evaluation prompt, in skeleton

You are evaluating a build against its specification. You did not write the build and should not assume it matches the spec.

Read: the design brief, the playing-god project entity in the vault (especially the standing caution and the invariants), the gate artefacts, and the logs.

Report, in this order: (1) which gates passed, with the measured numbers against the stated thresholds; (2) which mechanisms fired at rates materially different from the specification, with figures; (3) any invariant violated in implementation; (4) any constant the log evidence suggests is mis-set, with the evidence; (5) open questions the logs cannot answer, and what instrumentation would answer them.

Do not assess whether the output is good, whether any type of sound is performing better than another, or whether the search should be steered toward anything. If you find yourself proposing a constraint that would improve output by narrowing what can be produced, stop: that is the project's known failure mode and the proposal is wrong by construction.

15.4 On separate code briefs

One specification plus thin per-stage build orders. Not a second parallel description of the system.

A separate “code brief” restating the design creates two documents that must agree, and they will drift. The failure is silent: the coding agent follows the brief, the brief has quietly dropped an invariant, and nobody notices until the behaviour is wrong in a way nobody can trace. This document is already implementer-facing, which was the point of writing it this way.

What is genuinely useful is *sequencing*, not re-description. Each stage gets a short instruction of the form: **build §X-§Y of the specification and §14's logging for those sections; stop; run Gate Z; report the gate artefact.** A few lines plus pointers. No duplicated substance, therefore no drift surface.

Hand the coding agent this document and one stage instruction at a time. Do not hand it the whole build at once, because it will build the whole thing, and the gates exist precisely to prevent that.

16. DEFERRED

Indirect encoding (CPPN). Replacing explicit envelope nodes with a small network mapping (*slot index, time*) → (*amplitude, pitch*) would reduce the genome to a few dozen genes with unbounded phenotype detail, and would produce regularity across waves.

Not adopted for v1. Every indirect encoding buys compression by making some patterns cheap, which necessarily makes others expensive; in CPPNs the activation set determines which regularities are cheap — periodic functions produce repetition, Gaussians produce symmetry — and outputs carry a recognisable house style. Under the scoping rule in §2.1 this is not a literal truncation but it is the same defect in effect: some regions become expensive enough to be unreachable within budget.

Two further costs, both material. Indirect encodings have **low locality** through pleiotropy — one gene affects many phenotype elements, so a single change produces several qualitative changes at once — which is directly hostile to Gate 1b and §13.2. And adopting it would **discard most of the variation layer designed here**: with no per-wave gene blocks there is nothing for wave-intact slot-preserving inheritance, the duplication operator, the per-wave kill switches or the compatibility metric to operate on. §7 through §11 survive; §3's switch architecture and §6.3, §6.5, §6.6 and §6.8 do not.

A full decision analysis is held in the project entity rather than here. Should it be adopted later, the published path is HybriD (Clune et al.): evolve with the indirect encoding first to capture regularity, then switch to the direct encoding to handle irregularity — which is strictly better than choosing between them at the outset.

Complexity as a third archive axis. §7.4.

Archive culling below a global fitness floor. §7.3. Trades illumination for convergence.

Multi-user operation. The mission statement says “a user or users”, and pooling dwell across listeners has never been designed. Nothing is specified about deployment, hosting, synchronisation, or what `listener_id` does beyond being recorded.

Explicitly deferred, for a reason rather than by omission: the fitness signal is already high-variance from a single listener, and pooling across people adds a between-listener term that would need modelling as a listener random effect — which cannot be fitted without data that does not yet exist. The system is local-first in IndexedDB with no server.

The forward-compatible move is already in place at no cost: `listener_id` is recorded on every listen and every note, so pooling can be done retrospectively once there is enough data to model it. Do not build multi-user support now; do not drop the field either.

Predictor feature representation. §10 specifies the raw 6,102-vector as input. An MLP over a vector whose entries are mostly inert — muted waves, unused envelope nodes — is a poor representation, since the model must first learn which slots matter. Some feature engineering (per-wave pooling over active slots, or an attention over slots) is probably needed. Not designed, and not needed until Stage 4.

Explicit re-sampling of individuals. Superseded by deep cells (§7.2), which achieve implicit averaging at no additional listen cost. Would be reconsidered only if the deep grid proves unable to denoise within budget.

Rejected outright: corpus-based or concatenative synthesis (better sound, obtained by assembling recorded material, which abandons the premise); fitness-proportionate selection (§7.5); any constraint that improves output by narrowing what can be produced (§2.1).

APPENDIX — CONSTANTS

WAVE_SLOTS	= 64	GENES_PER_WAVE	= 95
GLOBAL_GENES	= 22	GENOME_SIZE	= 6102
P_ACTIVE_AT_INIT	= 0.03	MIN_ACTIVE	= 1
EXPRESSED_AT_INIT	≈ 163	SEED_BATCH	= 32 (σ = 0.2)
SAMPLE_RATE	= 44100	DESCRIPTOR_RATE	= 22050
KILL_FADE_MS	= 8	VIS_ENVELOPE_HZ	= 60
PITCH_RANGE_CENTS	= 0 .. 25100	PITCH_NODE_OFFSET	= ±9600
GAIN_OUT_DB	= -80 .. +6	ENV_LEVEL_DB	= -80 .. +24
GAIN_MOD	= 0 .. 32	PM_DEPTH_MAX	= 32
AM_DEPTH_MAX	= 8	ENV_MAX_NODES	= 8
PRE_WAIT_MAX_S	= 30	DURATION_S	= 0.0005 .. 120
TEMPO_BPM	= 30 .. 300	MID_WAIT_MAX_S	= 30
SIGMA_INIT	= 0.05	SIGMA_FLOOR	= 0.002
TAU_GLOBAL	= 0.0091	TAU_LOCAL	= 0.0801
P_SWITCH_FLIP_BASE	= 0.004	P_REROUTE	= 0.02
REROUTE_DEPTH_SCALE	= 0.05	P_NODE_COUNT	= 0.03
P_DUPLICATE	= 0.08	DUP_SIGMA_MULT	= 3.0
DUP_N_TARGETS	= 1 (p .75), 2 (p .20), 3 (p .05)		
DUP_TARGETS	= any of the other 63 slots, muted or active		
DUP_ARRIVAL_ATTEN	= 0.05 (gain_out, and inbound routing depths, only when overwriting an ACTIVE slot)		
MUTATION_FRACTION	= 0 .. 1		
ARCHIVE	= 16 x 16 = 256 cells, CELL_DEPTH D = 8		
AXIS_1	= temporal development (8-segment MFCC mean pairwise distance), log		
AXIS_2	= harmonicity (spectral flatness), log		
CROSSOVER_RATE	= 0.50		

N_PARTNERS = gene, 1 .. 8, init 1.4 (how MANY partners)
 PARTNER_INFLUENCE = gene, 0 .. 0.5, init 0.15 (how MUCH from partners, total)
 $D = 0.4*d_{switch} + 0.4*d_{active} + 0.2*d_{global}$ (each term in [0,1])
 D_MED = median D over random archive pairs, refreshed /100 listens
 PARTNER_KERNEL = $\exp(-D / (LAMBDA * D_MED))$
 LAMBDA = 0.25 (partner at $2*D_med$ is 55x less likely than at D_med)
 genealogical bonus REMOVED - genotypic only
 PARTNER_SAMPLING = proportional to kernel over ALL occupied cells; no threshold,
 no failure case, no fallback
 CELL_SELECT_ALPHA = 2.0 (rank-based, over cells)
 IN_CELL_SELECT_BETA = 2.0 (rank-based, among residents; parsimony breaks ties)
 PARENT_SCORE = $0.7 * r_{fitness} + 0.3 * r_{yield}$ (cell level)
 CELL_FITNESS = mean fitness of that cell's residents
 YIELD_SHRINKAGE_M = 5 YIELD_WINDOW = 50 offspring
 EVICTION = uniformly random resident; ignores fitness
 NEWCOMER_PROTECTION = 2 further arrivals; max half a cell protected; fitness-blind
 P_REPLAY = 0 (removed - deep cells make implicit averaging free)
 REPEAT_COOLDOWN_W = 30 listens REGEN_TRIES = 5
 DWELL_PER_GENOME = plain arithmetic mean of that genome's own listens;
 relatedness weighting applies BETWEEN individuals only
 PARSIMONY_TIE_BAND = 0.95 .. 1.05 (in-cell selection only)

 LINEAGE_DEPTH = 3 (global)
 DEPTH_WEIGHTS = 0.5 (self), 0.3 (depth 1), 0.2 (depth 2)
 each ancestor weight multiplied by inherited fraction
 CONTRIB_MAX_ANCESTORS = 8
 MIN_DWELL_S = 0.35 DWELL_CEILING = L (censored)
 IDLE_TIMEOUT_S = 90 UNATTENDED_WEIGHT = 0.25

 SERVO_WINDOW_X = 100 SERVO_THRESHOLD_T = 0.10
 SERVO_EXTEND_STEP = 1.5 SERVO_MIN_CHANGE = 0.05
 L_INIT_S = 60 L_FLOOR = 15 L_CEILING = 300

 PREDICTOR_ENSEMBLE = 5 HIDDEN = 3 x 256
 MIN_LISTENS_TO_START = 2000 AUTONOMOUS_ITERS = 1000
 LCB_K = 1.0 REGROUND_LISTENS = 60
 RHO_HEALTHY = 0.40 RHO_ABORT = 0.20

 LOCALITY_EPSILONS = 0.001, 0.01, 0.1, 0.5
 LOCALITY_CRITERION = $p_{90} < 0.20 * U$ at $\epsilon = 0.01$
 BEHAVIOURAL_LOCALITY = $p_{same} \geq 0.35$ AND $p_{near} \geq 0.70$ (Gate 2b, mutation-only)
 SWITCH_RATE_FORMULA = $0.004 * \min(1, 0.25 / J_{class})$